Onepoint

AI-Powered Newsletter Generator

Technical Brief & Handover Document

A. Original Brief Request (from CEO)

The following is the original request provided by the CEO, shared here for context so the technical team has the full background alongside the structured brief below.

> *"As CEO of Onepoint, help me provide my technical team with a brief for AI to automatically create a newsletter for multiple uses (internal to Onepoint, charity). The inputs can be provided via email or as documents (Word, Google Docs, or images) to a specific email address. There should be a standard newsletter design which can easily have the header and footer modified for internal Onepoint use, client use, or charity use. The output should then be the draft newsletter to be reviewed and sent."*

B. Structured Technical Brief

1. Objective

Build a system that automatically **drafts newsletters** from raw input material (emails, documents, images) submitted to a dedicated inbox. The system must support three newsletter contexts — **Onepoint internal, client-facing, and charity** — using one shared content engine and layout, with only the header/footer branding changing per context. Output is always a draft for human review before sending.

2. User Workflow (end-to-end)

- A contributor emails content to a dedicated address **(e.g. newsletter@onepoint.com**), either writing the update directly in the email body, or attaching source material: Word doc, Google Doc (link or export), image (e.g. photo of a printed flyer, screenshot, whiteboard photo), PDF, etc.
- The contributor indicates which newsletter the content is for (see Section 4 — routing).
- The system ingests, extracts, and structures the content.
- AI drafts newsletter copy (headline, summary, body sections) in a consistent tone/style.
- The system applies the correct template variant (internal / client / charity header & footer).
- A draft newsletter is generated and routed to a reviewer/editor for approval.
- On approval, it's sent (or exported/queued in the relevant sending tool — see Section 7).

3. Inputs to Support

Input Type	Handling Required
Plain email body text	Direct text extraction

Input Type	Handling Required
Plain email body text	Direct text extraction
Word documents (.docx)	Document parsing (text, headings, images)
Google Docs	Shared as link (requires API access) or exported/attached as .docx/PDF
Images (photos, screenshots, scans)	OCR + image captioning where OCR isn't enough (e.g. event photos with no text)
PDFs	Text extraction, OCR fallback for scanned PDFs
Multiple attachments in one email	Must combine multiple items into a single draft, or generate multiple sections/stories

Open question for the team: do we need to support combining multiple unrelated submissions into a single issue (e.g., the weekly digest), or is each email always one story/item? This significantly affects the design of ingestion (batching vs one-shot).

4. Routing Logic (which newsletter is this for?)

We need a reliable way to determine intended use. Options to evaluate:

- Separate email addresses per newsletter (**e.g. internal-news@, client-news@, charity-news@)** — simplest and most robust.
- Single address + keyword/tag in **subject line (e.g. [CHARITY])** — more flexible but relies on user compliance.
- AI-inferred routing based on content — higher risk of misclassification; not recommended as the sole method.

Recommendation: default to separate addresses (or subject tags as a fallback), with AI classification only as a secondary check/flag if the destination seems mismatched with the content.

5. Content Generation (AI processing)

For each submission, the AI should:

- Extract the core facts/message from the input (who, what, when, why it matters).
- Generate a suggested headline.
- Generate body copy in a consistent house style/tone (configurable per newsletter type — more formal for clients, warmer for charity, casual for internal).
- Identify and appropriately place any images supplied.
- Suggest a section category if the newsletter has recurring sections (e.g. "Team News," "Client Wins," "Upcoming Events," "Volunteer Spotlight").
- Flag content that seems incomplete, ambiguous, or requires the contributor to confirm details (dates, names, figures) rather than guessing.
- Never fabricate facts, figures, quotes, or names not present in the source material — this needs to be an explicit guardrail.

6. Template & Branding System

- One master layout (structure, fonts, spacing, section types) shared across all three newsletters.
- Three swappable header/footer sets, each with its own logo/branding, colour accents, sign-off/footer text (e.g. legal disclaimers for clients, charity registration number for charity, internal contact info for staff), and optionally a default tone-of-voice setting tied to the template.
- The template should be built so a non-technical person can update header/footer assets later without needing a developer (e.g., stored as editable config/assets, not hardcoded).

Suggested build approach: HTML/CSS email template (most portable, easy to theme) rather than a design-tool-only format, so it can plug into standard email sending tools.

7. Output & Review Step

- Output is always a draft, never auto-sent.
- The draft should be delivered somewhere reviewable — e.g., emailed back to a designated editor/approver, posted to a shared drafting doc/tool, or pushed into the email platform used for sending (e.g., as a draft campaign).
- The reviewer should be able to edit copy, swap images, and approve/reject before sending.
- Once approved, sending can be manual (reviewer hits send) or, in the long term, automated to the appropriate distribution list.

8. Key Technical Components to Scope

- Inbound email handling — mailbox or API (e.g. a dedicated inbox with an email-parsing webhook).
- Document/image parsing — text extraction (docx, PDF), OCR (images/scanned docs), possibly image understanding for photos without text.
- AI generation layer — LLM call(s) to structure and draft copy, with prompt/style controls per newsletter type.
- Template engine — HTML/CSS templating with variable header/footer blocks.
- Review/approval mechanism — where drafts land and how approval is captured.
- Sending integration (optional, phase 2) — connection to whatever tool is used to distribute the newsletter (e.g. Mailchimp, Outlook, internal comms tool).
- Access control — who can submit content, who can approve, and whether client-facing content needs stricter review than internal.

9. Non-Functional Considerations

- Data handling: client and charity data may have different confidentiality/compliance requirements — flag if legal/compliance needs to weigh in.
- Brand consistency: client-facing output should have a tighter tolerance for tone/errors than internal.
- Auditability: worth logging what was submitted vs. what was generated, for accountability given AI-generated content.
- Fallback: if AI extraction fails or confidence is low, the system should flag for manual handling rather than silently producing a poor draft.

10. Suggested Phasing

Phase	Scope
Phase 1	Single newsletter type (e.g. internal), text-only inputs, manual review, HTML template with one header/ footer.

Phase	Scope
Phase 1	Single newsletter type (e.g. internal), text-only inputs, manual review, HTML template with one header/ footer.
Phase 2	Add document/image ingestion (OCR), add second and third newsletter types with swappable branding.
Phase 3	Add routing automation, approval workflow, and sending integration.

11. Open Questions for the Technical Team

- What's our preferred stack for email ingestion (existing mailbox + webhook vs. dedicated email API)?
- Do we have an existing sending tool (Mailchimp, Outlook, etc.) that this should integrate with, or is manual send acceptable initially?
- Who are the designated reviewers/approvers for each of the three newsletters?
- What cadence do we need (weekly, monthly, ad hoc)?
- Any regulatory/compliance constraints on the charity newsletter (e.g. registered charity disclosure requirements)?